

Network and Wi-Fi Security Scanner

IT359 Cybersecurity Final Project Report

By: Devon Mann and Dawson Raines

Fall 2025

1. Introduction & Purpose

1.1 Problem Statement

Network security is a critical concern for organizations and individuals alike. As networks grow in complexity and the number of connected devices increases, identifying vulnerabilities and potential security risks becomes increasingly challenging. Traditional security assessments often require expensive commercial tools or significant manual effort to conduct comprehensive network analysis.

This project addresses the need for an accessible, integrated security assessment tool that combines multiple scanning capabilities into a single, cohesive framework. As stated in our project README, the purpose is to design and implement a network and Wi-Fi security scanner that can identify potential vulnerabilities and provide insights into overall network security. The system leverages existing security tools while integrating custom logic through Python scripting to create a cohesive security assessment framework.

1.2 Project Goals

As outlined in our initial project proposal, the primary objectives were:

1. Develop a Python-based network scanner integrated with Nmap for host discovery, port scanning, and service enumeration
2. Implement Wi-Fi assessment capabilities to analyze wireless security strength and encryption standards
3. Design a lightweight anomaly detection system for real-time network traffic monitoring
4. Integrate vulnerability scanning to identify common security weaknesses
5. Deliver a functional proof-of-concept security toolkit by the end of the semester

1.3 Core Components

The project was designed around four core components as specified in our proposal:

1. **Network Scanner:** Utilizing Nmap with Python integration to scan local networks for active hosts, open ports, and running services, with custom detection logic for OS details and service versions
2. **Wi-Fi Security Assessment:** Using Aircrack-ng or alternative tools to evaluate Wi-Fi security configurations, encryption standards, and potential vulnerabilities in wireless protocols
3. **Network Anomaly Detection:** A monitoring system to analyze network traffic and detect unusual patterns that may indicate unauthorized access or suspicious activity
4. **Vulnerability Scanning:** Incorporating OWASP ZAP or similar tools to test for common web application vulnerabilities such as SQL injection, XSS, and misconfigurations

1.4 Scope

This security scanner was developed and tested on a local/home network environment as specified in our proposal. The tool is designed for use in authorized security assessments where users have explicit permission to scan. It serves as both a practical security tool and a learning opportunity to gain hands-on experience with network security, vulnerability analysis, and Python-based automation.

2. Technical Implementation

2.1 System Architecture

The application follows a modular architecture with separate scanner modules that can operate independently or in coordination. The codebase is organized into distinct Python modules: `network_scanner.py` for host and service discovery, `vulnerability_scanner.py` for CVE lookups, and `main.py` which coordinates the scanning modules and provides the user interface. This design pattern allows for clean separation of concerns and facilitates testing and maintenance.

2.2 Network Scanner Module (`network_scanner.py`)

The `NetworkScanner` class in `network_scanner.py` serves as the foundation for network discovery operations. It utilizes the `python-nmap` library, which provides a Python interface to the Nmap security scanner. The class initializes with an `nmap.PortScanner()` instance, a `scan_results` dictionary for storing findings, and an `is_scanning` flag for scan control.

Basic Scan Method

The `basic_scan()` method performs a ping sweep using the `'-sn'` argument across the specified target range (defaulting to `'192.168.1.0/24'`). For each host where `nm[host].state()` returns `'up'`, the scanner records the IP address, hostname via `nm[host].hostname()`, and status. Results are stored in `self.scan_results['basic_scan']` with a timestamp, target range, and list of active hosts.

Detailed Scan Method

The `detailed_scan()` method performs comprehensive analysis using the scan arguments `'-sS -O -sV --version-intensity 5'`. This enables SYN scanning (`-sS`), OS detection (`-O`), and thorough service version detection (`-sV` with intensity 5). For each active host, the method extracts OS guesses from the `'osmatch'` key and iterates through all protocols and ports to collect port number, state, service name, version, and product information. Results are structured with full host details including the ports array.

Additional Methods

The module includes `scan_specific_host()` which wraps `detailed_scan()` for single-host scanning, `save_results()` which serializes `scan_results` to JSON in the `data/` directory with timestamps, and `stop_scan()` which sets `is_scanning` to `False` for graceful termination. The scanning loop checks this flag to allow interruption of long-running scans.

2.3 Vulnerability Scanner Module (`vulnerability_scanner.py`)

The `VulnerabilityScanner` class extends scanning capabilities by correlating discovered services with known vulnerabilities. Rather than OWASP ZAP as

originally proposed, the implementation uses the Vulners.com API, which aggregates vulnerability information from CVE, NVD, and vendor advisories. The class maintains an optional `api_key`, `scan_results` dictionary, `is_scanning` flag, and its own `nmap.PortScanner()` instance.

Host Scanning Process

The `scan_host()` method first performs service detection using `'-sS -sV --version-intensity 5'` arguments. For each discovered service with valid product and version information, it calls `check_service_vulnerability()` to query the Vulners API. The method constructs a Lucene search query combining product and version strings, posts to `'https://vulners.com/api/v3/search/lucene/'` with a 1-second rate limit delay, and processes up to 10 results per query.

Vulnerability Filtering and Risk Assessment

The `_is_relevant_vulnerability()` method filters results by checking if product terms appear in the vulnerability title or description. The `_calculate_risk_level()` method classifies hosts based on maximum CVSS scores: 'Critical' for 9.0+, 'High' for 7.0-8.9, 'Medium' for 4.0-6.9, and 'Low' otherwise. Each vulnerability record includes `id`, `type`, `title`, `truncated description`, `cvss_score`, `published date`, `references`, and `bulletinFamily`.

Network-Wide Scanning

The `scan_network()` method performs discovery across a target range using `'-sn'`, then iterates through active hosts calling `scan_host()` for each. The `generate_report()` method produces a formatted text summary showing each host's IP, risk level, vulnerability count, and top 3 findings with CVSS scores.

2.4 Dependencies and Requirements

As specified in `requirements.txt`, the project depends on:

- **python-nmap==0.7.1:** Python bindings for Nmap providing programmatic access to scanning functionality
- **Pillow==10.0.0:** Python Imaging Library for GUI components

Additional implicit dependencies include:

- **requests:** Used in `vulnerability_scanner.py` for Vulners API communication
- **Nmap:** Must be installed on the system for `python-nmap` to function

- **socket:** Standard library module imported in vulnerability_scanner.py

3. Justification & Analysis

3.1 Technology Selection Rationale

Python as the Development Language

As specified in our proposal, Python was chosen as the primary development language. Its extensive library ecosystem provides robust support for network programming and security tools. The python-nmap library offers a mature, well-documented interface to Nmap's capabilities. Python's readability facilitated collaborative development between team members, and its cross-platform compatibility ensures the tool can run on various operating systems.

Nmap for Network Scanning

Nmap was specified in our proposal as the core scanning engine. It is the industry-standard tool for network discovery and security auditing, offering proven reliability and a comprehensive feature set. The scanning techniques used in our code (SYN scan, service detection, OS fingerprinting via -sS, -sV, -O flags) are well-documented and produce actionable results. The python-nmap wrapper provides clean access to these capabilities.

Vulners API vs. OWASP ZAP

Our original proposal specified OWASP ZAP for vulnerability scanning. During implementation, we pivoted to the Vulners API because OWASP ZAP focuses on web application testing requiring running web servers, while Vulners better suited our network-level vulnerability correlation needs. Vulners aggregates CVE, NVD, and vendor advisories, supports flexible Lucene queries by product/version, and provides a free tier adequate for educational use.

3.2 Architectural Decisions

Modular Class Design

The decision to implement NetworkScanner and VulnerabilityScanner as separate classes in distinct files provides clear benefits. Each module can be imported and tested independently. The VulnerabilityScanner maintains its own nmap instance, allowing it to perform service detection without depending on NetworkScanner results. This supports future expansion—the planned Wi-Fi and anomaly detection modules can be added as additional classes without modifying existing code.

Scan Control Pattern

Both scanner classes implement the is_scanning flag pattern. The scanning loops check 'if not self.is_scanning: break' to enable graceful termination via the stop_scan() method. This recognizes that network scans can be time-consuming, especially detailed scans with OS and version detection, and users may need to interrupt operations while preserving partial results.

Result Storage Strategy

Both modules store results in `self.scan_results` dictionaries and implement `save_results()` methods that serialize to JSON with timestamps. Files are saved to the `data/` directory with naming patterns like `'scan_results_YYYYMMDD_HHMMSS.json'` and `'vuln_scan_results_YYYYMMDD_HHMMSS.json'`. This enables historical comparison and audit trails of scanning activity.

3.3 Deviations from Original Proposal

Several adjustments were made from our original proposal:

- **Vulnerability Scanning:** Shifted from OWASP ZAP to Vulners API for better alignment with network-level rather than web-application-level assessment
- **Wi-Fi Assessment:** The Aircrack-ng integration specified in the proposal was not completed in this version due to hardware driver requirements
- **Anomaly Detection:** The Scapy-based traffic analysis with Pandas/Matplotlib visualization was scoped for future development
- **Timeline Adjustments:** Focus was placed on delivering robust network and vulnerability scanning rather than partial implementations of all four components

3.4 Code Quality Considerations

The code includes comprehensive docstrings for each method explaining functionality. Exception handling with `try/except` blocks ensures scan failures don't crash the application—exceptions are caught, `is_scanning` is reset to `False`, and errors are re-raised for caller handling. The vulnerability scanner implements rate limiting (`time.sleep(1)`) to respect Vulners API policies and avoid service disruption.

3.5 Limitations and Future Work

Areas identified for future improvement include:

- **Wi-Fi Integration:** Implementing the originally proposed Aircrack-ng integration for wireless security assessment
- **Anomaly Detection:** Adding Scapy-based packet capture with Pandas/Matplotlib visualization as proposed
- **False Positive Reduction:** The current `_is_relevant_vulnerability()` uses simple string matching; more sophisticated NLP-based correlation could improve accuracy
- **Missing Dependencies:** The `requests` library used in `vulnerability_scanner.py` should be added to `requirements.txt`

- **Enhanced Reporting:** Expanding generate_report() to produce PDF/HTML output with remediation guidance

4. Conclusion

4.1 Project Summary

This semester-long project successfully developed a network security scanning framework consisting of `network_scanner.py` and `vulnerability_scanner.py` modules. While not all originally proposed components (Wi-Fi assessment, anomaly detection) were fully implemented, the delivered codebase provides functional host discovery, service enumeration, and vulnerability assessment capabilities with a modular architecture designed for future extension.

4.2 Key Achievements

The project accomplished the following:

1. Implemented NetworkScanner class with `basic_scan()` for host discovery and `detailed_scan()` for comprehensive port/service/OS enumeration
2. Developed VulnerabilityScanner class integrating Vulners API for real-time CVE lookups based on detected services
3. Created CVSS-based risk assessment with Critical/High/Medium/Low classification
4. Built modular architecture with independent scanner classes supporting future component additions
5. Implemented JSON result storage with timestamps and text-based report generation

4.3 Learning Outcomes

Through this project, we gained practical experience with network security concepts including TCP/IP scanning techniques, service fingerprinting, and vulnerability databases. Working with `python-nmap` provided understanding of how security tools interface with underlying system utilities. Integrating the Vulners API taught REST API consumption, rate limiting, and result filtering. The development process reinforced software engineering practices including modular design, exception handling, and documentation.

4.4 Future Development

The modular architecture positions the codebase for future enhancement. Priority additions include the Wi-Fi security module using Aircrack-ng as originally proposed, the anomaly detection system using Scapy with Pandas/Matplotlib visualization, GUI improvements leveraging the Pillow dependency, and enhanced reporting capabilities. The existing NetworkScanner and VulnerabilityScanner classes provide a solid foundation that these additions can build upon without requiring significant refactoring.

— End of Report —